MATERIALIZE — code architecture

What this document is. The authoritative map of the *code*: what the layers are, which way dependencies run, which parts are protected and why, and where new work belongs. It is the "why is it shaped like this" companion to the file-by-file tour in `MATERIALIZE.md` §5–§9 (physics and usage) and to the operative rules in the project `CLAUDE.md` §2 (the short form used while working).

On any conflict: `CLAUDE.md` is the operative rule, this document is the explanation, and code wins over both. Last reconciled with the tree: **2026-08-15**, after the A-7b re-layering.

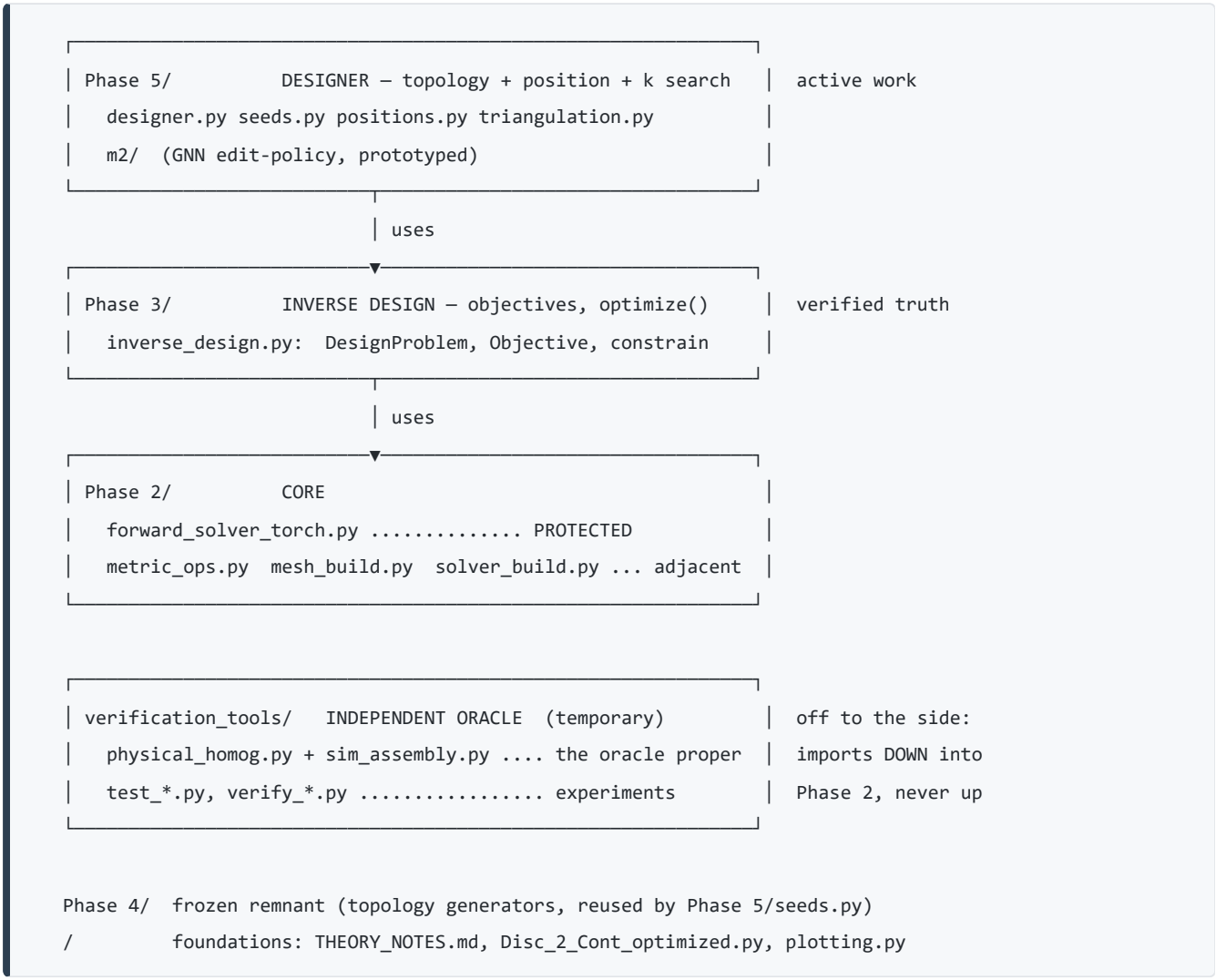
---

1. The one-paragraph version

MATERIALIZE does inverse design of mechanical metamaterials. A **fast differentiable solver** maps (topology, positions, stiffness) → effective elastic response; an **inverse-design layer** runs gradients through it; a **designer** searches over topologies on top of that. Beside all of it — not inside it — sits an **independent simulation oracle** that exists to catch the solver being wrong. The architecture is organised so that (a) dependencies point inward toward the solver, and (b) the oracle shares no code with the thing it checks.

2. Layers

Dependencies point **inward**. A layer may use anything below it and must not know about anything above it. Phase numbering is historical, not a ladder: the live arc is Phase 2 → Phase 3 → Phase 5.



Phase 2 — the core

module	role	depends on
`forward_solver_torch.py`	**PROTECTED.** `ElasticSolver` : (k, ℓ_0, geometry) → C(s), C_eff, ν, E, W. Differentiable; sparse-KKT adjoint above 600 triangles	torch
`metric_ops.py`	the NumPy side of the solver's own conventions: `vec3` (Voigt [xx,xy,yy]), `tri_metric_change` (Δg = $F^TF - \bar{g}$), `bare_tensor` (A(s))	numpy
`mesh_build.py`	periodic and open mesh construction: `build_geometry` , `set_VD` , `kkt_from_tri_bond` (the C1 constraint topology), `clean_tri` , `build_open_mesh`	numpy
`solver_build.py`	`make_solver` / `_mount` : an `ElasticSolver` from a periodic geometry dict (unwrapped edges + PBC constraints)	torch, the core

The three lower modules are **core-adjacent**: same layer, same gate, *not* the protected file. `DesignProblem` 's constructors are built directly on them, so treat them as verified truth.

Phase 3 — inverse design

`inverse_design.py` owns `DesignProblem` (periodic / open / from_geo), `Objective` , `constrain` , `optimize` (L-BFGS through the solver), `validate` . `verifications/` holds end-to-end design→simulate→check demos and

the shared harness `_common.py`.

Phase 5 — the designer

M1 search over topology, node positions and k, built on Phase 3. M2 is the prototyped GNN edit-policy. This is where active work happens.

verification_tools — the oracle

`physical_homog.py` (nodal relaxation → virial stress / energy-Hessian homogenisation) plus `sim_assembly.py` (`assemble_K_faff`, the assembler it is handed). Everything else in the directory is an **experiment script, not a library**.

3. The two load-bearing invariants

(i) Protected core

`forward_solver_torch.py` is not modified without an explicit request and approval. Gate: `Phase 2/test_forward_solver.py` **and** the physical `verify_*` suite. A core change is not done until dependents are re-verified and the blast radius checked.

(ii) The oracle shares no code with the design path

This is the subtle one, and it is the reason for the whole layout.

The solver is the *fast path*; the simulation is the *truth it is checked against* while the framework is being validated. That check is only worth anything if the two are genuinely independent. If the oracle imports any part of the design path, "checked against the sim" quietly degrades into checking the code against itself.

This is not hypothetical. In 2026-08 a defect in the homogenisation contraction (the shear channel, `C_xyxy` over-stiff by up to 90%) survived for months because the one tensor-level check routed the simulation's result back through the *solver's own* contraction. See `documentation/shear_channel_defect.md`.

Consequences, all deliberate: - `physical_homog.py` depends on nothing but NumPy/SciPy — it keeps its **own** copies of `DELTA` and `MODES` rather than importing them. **That duplication is intentional; do not "fix" it.** - The oracle is a named, self-contained pair (`physical_homog` + `sim_assembly`) so its boundary is checkable rather than a matter of opinion. - Tests and verification scripts *should* import the oracle — that is their job. `Phase 2/test_forward_solver.py` [7] imports it precisely so the comparison has an independent side. The dependency rule is about **production** code, not about who may run a check.

"Independent" is a property of a specific routine, not of the directory. The operative coverage table lives in `CLAUDE.md` §3 and is kept current; the shape of it is:

genuinely independent (no solver code in the path)	NOT independent (shared contraction)
`energy_C`, `virial_C` — bulk tensor	`_common.sim_per_triangle_C6` — per-triangle
`energy_C_per_triangle` — per-triangle `C(s)`	`_common.region_phys_C6` — regional
`energy_C_region` — regional `C`	
`virial_nuE` / `energy_nuE` — bulk ν, E	

The right column is not wrong, it answers a different question: it validates the spatial *pattern* and the design→realise→simulate loop. It cannot validate the homogenisation, because it *is* the homogenisation. Gated by `test_forward_solver` [7] (bulk) and [8] (per-triangle and regional); [8] closed audit A-9 on 2026-08-15.

4. The dependency rule, and how it is enforced

No design/production module in Phase 2 or Phase 3 may import from `verification_tools/`.

The reason is concrete: that layer is *temporary and retireable* (it is retired once the solver is trusted). A design layer rooted in it could not survive its own validation succeeding.

Enforcement is structural, not just conventional: `verification_tools/` is **not on** `Phase 3/inverse_design.py`**'s** `sys.path`, so the import would fail rather than silently work.

History (audit A-7b, fixed 2026-08-15). This had been inverted. `inverse_design.py` imported its mesh construction, its constraint topology and its **solver construction** from four scripts inside `verification_tools/` — including `make_solver` / `_mount`, which build the `ElasticSolver` itself. Lifting them into Phase 2 is what created `metric_ops` / `mesh_build` / `solver_build`. Full account: `Phase 3/verifications/relayer_a7b_out/RELAYER_A7B.md`.

Known remaining violation of the same kind (A-7c, open): `Phase 3/verifications/_common.py` sits in a *verifications* directory but supplies mesh constructors and persistence to all of Phase 5. Its own upward dependency is gone, but its position is still wrong.

5. Other standing rules

- **Compute is separated from I/O and rendering.** Pure compute returns data; designed networks are **saved** (`save_network`), and figures **load** them — never re-optimise at plot time, since random restarts make that non-reproducible.
- `plotting.py` **(repo root) is the single source of truth for figures.** Import its primitives; never roll a one-off `plot_*`. A missing primitive is added *there*.
- **Config is the single source of truth**; no scattered magic numbers.
- **Reproducibility:** every artifact traceable to (commit, config, seed). Randomness is isolated to explicit injection points — a seeded generator passed down, never a global RNG.
- **Artifacts live per-phase with provenance**; scratch stays disposable and out of the repo.

> **Stale artifacts are a real trap.** `verification_tools/plots/dg_solver_sweep_20x20.npz` looks like a reference for ν(η)/E(η) but predates both the physical-units change (its E is in internal units, low by the factor 18.4752 = (2/√3)/0.0625) and the switch away from the area-weighted average (so its ν is biased at η>0, exactly in the disordered regime it covers). Reproducing its numbers today would mean the corrections had been undone. Check provenance before treating any stored array as a target.

6. Where new work goes

you are adding…	it belongs in
a solver capability	`Phase 2/` — protected file only with approval + re-gate
a shared geometry/metric helper used by ≥2 layers	`Phase 2/metric_ops.py` or `mesh_build.py`
an objective, constraint, or optimiser feature	`Phase 3/inverse_design.py`
a designer / search / learned-model feature	`Phase 5/`
an independent physical check	`verification_tools/` — and it must share no code with the design path
an experiment	the relevant phase's `verifications/`, with outputs + a results `.md` under `results/`
a figure	`plotting.py` primitives; the script lives with its experiment

Anything that is *reused* (a second consumer appears) gets factored into one shared, tested module. Anything that is *depended upon as a truth* gets protected — changed only with approval.